

How Tezos blockchain can meet IoT?

Quang Huy DO^{*†}, Sami SOUIHI[†], Van Van TONG^{*†}, Hai Anh TRAN^{*}, and Skander MHADHBI[‡]

^{*}School of Information and Communication Technology, Hanoi University of Science and Technology, Vietnam

[†]LISSI-TincNET Research Team, University of Paris-Est Creteil, France

[‡] Texpect/TezosTNA

Email: huy.dq184271@sis.hust.edu.vn, sami.souhi@u-pec.fr, vantv@soict.hust.edu.vn,
anhth@soict.hust.edu.vn, sami.souhi@u-pec.fr, skander.mhadhbi@gmail.com

Abstract—Nowadays, blockchain, revolutionary technology with high security and decentralization is widely applied in many industry segments including healthcare, cryptocurrency and so on. Despite its benefit, it is a challenge for blockchain to effectively operate in large-scale networks without compromising security and decentralization properties due to low throughput. Therefore, in this paper, we consider sharding-based blockchain as a solution to overcome this limitation. The objective is to split the network into different shards to process transactions in parallel and reduce quorum size to reach consensus quickly. However, existing sharding-based blockchain networks implement a consensus algorithm for both main and sub-shards, so it is ineffective with different purposes (e.g., to improve security, reduce resource consumption, etc.). Hence, we propose a system with heterogeneous consensus algorithms in sharding-based blockchain to meet different requirements. Experimental results show that the proposed approach improves up to 54 percent of throughput in comparison with benchmarks.

Index Terms—Blockchain, Tezos, Sharding, Heterogeneous consensus, Tenderbake, Emmy plus.

I. INTRODUCTION

In recent years, Blockchain technology has drawn much attention in many industry segments such as healthcare [1], cryptocurrency, Internet of Things (IoT) [2] and so on. These segments take advantage of blockchain properties, including transparency, immutability, auditability, and decentralization. Despite these benefits, one of the most significant disadvantages of blockchain is its limited throughput. For example, Bitcoin can process seven transactions per second, while Tezos processes 40 transactions per second. These throughputs are still out of all proportion of what could be generated by an IoT network with more than 50 billion nodes. Consequently, applying blockchain to large-scale networks faces many problems. Therefore, in this paper, we have tried to answer this question by modifying the consensus algorithm of Tezos blockchain to make it more scalable.

There are two kinds of solutions to deal with the scalability problem of blockchain: on-chain and off-chain [3]. On-chain solutions (e.g., bigger blocks, directed acyclic graph, sharding, etc.) refer to modifications on the blockchain (e.g., changing blockchain structure), while off-chain solutions refer to mechanisms outside of the blockchain (e.g., Trinity, Raiden network, etc.). According to related work, [3], [4], sharding is one of the most promising solutions for the scalability problem. The main idea of sharding is to divide the network into different subsets,

called shards. Each shard processes some transactions rather than the entire network processing all transactions. Therefore, it processes transactions in parallel and reduces quorum size to reach consensus speedily.

However, current sharding-based blockchain networks adopt a static consensus algorithm for their shards. For instance, Elastico [5] implements Proof-of-Work (PoW), and Rapid-chain [6] adopts a synchronous consensus algorithm based on Byzantine Fault Tolerant (BFT). These systems have different requirements (e.g., improving security, reducing resource consumption, etc.). For example, PoW can be taken into account to ensure the security property in transactions related to cryptocurrencies, while BFT can be adopted to optimize resource consumption in other transactions. In this paper, we discussed the advantage of applying a sharding-based system in the Tezos blockchain to make it more applicable in the IoT domain. The main contributions of this paper are described as follows:

- The first contribution is to experiment with the classical sharding approach in the Tezos Blockchain
- The second contribution consists of a variant of the sharding method based on introducing a "main shard" in charge of the global consensus.
- We also try to analyze the relevance of introducing a heterogeneous consensus algorithm approach in the sharding-based blockchain, which aims to implement two consensus algorithms for the sub shards and the main shard to meet different requirements.
- Finally, we present a thorough analysis of the performance of these different solutions.

The remaining of this paper is structured as follows. Section II provides background on Tezos Blockchain and its consensus algorithms containing Emmy plus and Tenderbake. Section III describes our system overview of sharding-based blockchain. Section IV deeply explains how we implement our sharding-based algorithms. The performance analysis for sharding-based blockchain networks is carried out in Section V. Finally, the paper concludes with section VI which highlights our future work.

II. BACKGROUND: THE TEZOS BLOCKCHAIN

Tezos [7] is a public and permissionless blockchain. It has been widely adopted due to its distinctive features such as its self-amending mechanism, its usage of a 'Proof-of-Stake'

based consensus algorithm, and its newly proposed algorithm Tenderbake. One of the significant advantages of Tezos is that it does not require a hard fork. Performing updates in this way accelerates innovation and makes this blockchain the ideal solution for testing new approaches and getting them accepted more quickly in the community, especially since updates can be done at a high frequency without disrupting the system. From a practical point of view, all Tezos bricks can be modified, including the amendment mechanism itself. The latest proposal, called Ithaca, replaced the Tezos consensus algorithm, the "Emmy family" (a PoS algorithm), with a new consensus algorithm known as Tenderbake (a PBFT algorithm).

A. Emmy plus

"Emmy plus" [8] is one of the consensus algorithms of the Emmy family, which are technically proof-of-stake consensus algorithms. A node in the "Emmy plus" consensus algorithm maintains its id, priority, set of endorsements, and a list of blocks. The basic idea of Emmy plus is to have each node bake after a certain delay and skew the delays according to the priority and number of endorsements a node has. Namely, the higher the priority of a node and the higher its number of endorsements, the sooner it gets to bake.

In detail, at the beginning of a level, a validator checks if it is time to bake; if it is, the validator bakes a block and broadcasts its chain with the block as a new head and its endorsement, which says, "I validator x endorse the block b". When receiving a new chain, validators: check that the chain is valid and longer. If so, replace the current chain with it, set the priority with the level of the current head, set the endorsements to be empty, and finally, if the node is an endorser at this level, send an endorsement. If validators receive endorse messages, it ignores them if they have already been stored. Otherwise, check that it is a valid endorsement and store it.

B. Tenderbake

Tenderbake [9] is a classical BFT-style algorithm. A subset of nodes determines a block for each level. This subset is called the committee per level. For each level, time is divided into a series of turns. The process continues to turn by turn until a node has decided on a block.

As shown in Fig. 1, Tenderbake automaton has three distinct states, which correspond to the possible phases of the consensus algorithm: NP for Non-Proposer, CP for Collecting Preendorsements, and CE for Collecting Endorsements.

In detail, from the state NP, a validator is randomly chosen as the proposer for the next level and will make a new block and propose the new head. If the proposed chain and message are valid for a given proposal message, and the chain is "better", it updates the state to CP and adds a self-preendorsement. If a validator receives a preendorse message, it checks if the pre-endorsement is valid and matches the current chain. If its state is CP, it adds this preendorse message and checks if the Preendorse quorum certificate (PQC) is reached. We say a node reaches PQC if it has already received more

than 2/3 preendorse messages. If so, change the node state to CE and endorse the new head. Finally, for a given endorse message, a validator checks the validation of endorsement and matches its chain. If everything checks out, it adds the endorsement to our collection and checks the Endorse quorum certificate (EQC) is reached. EQC shows if receiving more than 2/3 endorse messages. If so, the validator decides this endorsed block and go to next level.

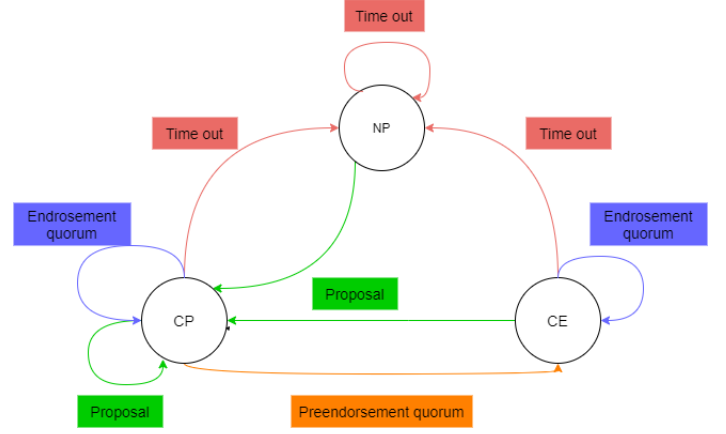


Fig. 1: Tenderbake automaton.

III. A SHARDING-BASED SYSTEM FOR TEZOS

In order to allow the Tezos blockchain to meet the needs of an IoT network in terms of the number of transactions, we propose introducing a sharding-based system as a solution to overcome the scalability problem.

Originally, sharding [10] [11] is a type of database partitioning technique that separates a huge database into much smaller, faster, more easily managed parts called data shards. The key idea of sharding in the blockchain is to split the network into smaller committees. Each can process a disjoint set of transactions (or a "shard"). Specifically, the number of committees grows linearly in the total computational power of the network. Moreover, each committee has a reasonably small number of members, so they can run a classic Byzantine consensus protocol to decide their agreed set of transactions in parallel. As shown in Fig. 2, communications play an essential role in sharding blockchain systems. Nodes in the same shard only need to execute intra-shard communication most of the time and send some essential information to the shard coordinator. The coordinator is usually responsible for cross-shard communication and intra-shard consensus, and each shard has at least one coordinator. This paper focuses on intra-committee consensus, which defines a standard consensus protocol each node within a committee runs.

Our proposal, in this paper, is divided into 2 types of approaches. The first one is a sharding approach without the main shard, we only have multiple sub shards running Tenderbake, and nodes from different shards exchange messages directly as described in Fig.3. We call this first approach as **Classical Sharding without main shard**. The second approach with the

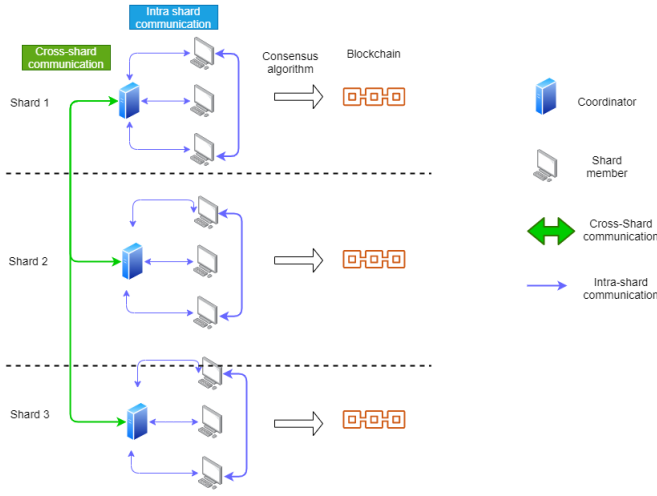


Fig. 2: Sharding blockchain Systems.

main shard is described in Fig. 4. In this example, we have nine nodes originally with id from 1 to 9 and we consider three shards. In shard 1, the leader is node 3; for shard two, the leader is node 6; and finally, the leader of shard 3 is node 9. These three leaders will form a main shard. And these 3 leaders will also maintain 2 states with following structure: $[\text{state_sub_shard}, \text{state_main_shard}]$. This approach can be thought of with the same consensus algorithm for the shards or by having one algorithm for the main shard and another for the other shards. One can even imagine a different algorithm for each shard.

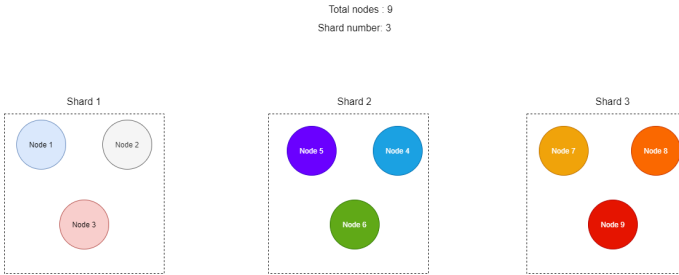


Fig. 3: System model for Classical Sharding without main shard.

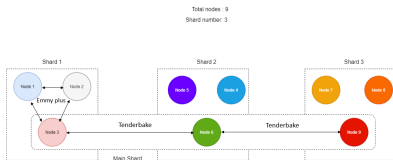


Fig. 4: System model for Sharding approach with main shard.

IV. IMPLEMENTATION

In this section, we introduce the implementation of our sharding approaches. We defined our system as a set of **nodes**

each with a unique **node_id**. Each node has a **node_state**, and some messages can be sent between nodes.

From the perspective of a node, it sits around until it receives an **event**, at a specific time, then processes the event to update its state and produces a new state and a list of **effects**. An event is either a wake-up call for a particular time or a received message. An effect is either a "self wake-up" call for some future global time or a message broadcast to every node except itself.

All of the logic of a node is handled by an **event_handler**. We say an **iteration** is each time a node handles an event.

The system is started by each node receiving a magical "wake up" event at time 0. The system progresses by **iterations**. Each iteration runs an event on a node, collects the effects and converts them to events.

Traditional implementations of sharding mechanisms generally support a single consensus algorithm. To apply sharding and run different consensus algorithms for different shards, we define a parameter called **number_of_shard** and the **shard_size** will be result of euclidean division between **number_of_nodes** and **number_of_shard**. Then nodes are divided into smaller groups called **shards**. Each shard maintains the same number of nodes as the shard size. Moreover, in a shard, beside a proposer in each level and round, we also choose a fixed **Leader**. **Leader** from each shard maintains two states: a state of sub shard (the initial shard) and a state of the main shard formed by leaders in all sub shards. In each shard, nodes then handle event messages sent by other nodes to reach the consensus state of a new block. In the following, we will present the details of the implemented approaches.

A. Classical Sharding without the main shard

In this approach, each shard can produce blocks parallel and concurrently, which improves protocol throughput.

a) **Node state**: The node state of **Classical Sharding without the main shard** is showed in Alg. 1.

Algorithm 1 Node state of Classical Sharding without the main shard

```

node_state:
id = node_id,
shard_id = node_id / (total_nodes / num_shards),
chain = [],
proposal_state = No_proposal

```

b) **Event handler**: With **Classical Sharding without main shard**, "propose phase" is handled as described in II-B. For the second phase of the protocol, "preendorse", nodes broadcast their preendorse message to every node, including nodes from other shards. After receiving the preendorse message, nodes will check if the message is valid and broadcast the "endorse" message. As a result, a block can only be decided if EQC is greater than or equal to 2/3 of total nodes. Finally, for event wake up, nodes behave according to Algorithm2.

Algorithm 2 Wake up event algorithm of Classical Sharding without main shard

```

if is_proposer then
  broadcast proposal;
else
  increment round;
end if

```

B. Classical Sharding with the main shard

The most significant disadvantage of the **Classical Sharding without the main shard** is that nodes that want to communicate with nodes from other shards must send messages directly, which causes message overhead. So we propose implementing the main shard formed by leaders of each shard, as shown in 4. When nodes want information from nodes of different shards, nodes can get that from the Leader of its shard. We can reduce many messages sent between shards by forming the main shard.

a) *Node state*: We define two states - the state of the main shard and the state of the sub shards. The leaders must maintain sub-shards and main shard states while the normal nodes keep their sub-shard states. In detail, the state of nodes is defined as in Alg. 3.

Algorithm 3 Node state of Classical Sharding with the main shard

```

main_shard_state:
id = node_id,
shard_id = -1,
chain = [],
proposal_state = No_proposal
sub_shard_state:
id = node_id,
shard_id = node_id / (total_nodes / num_shards),
chain = [],
proposal_state = No_proposal
node_state ← []
if is_leader then
  node_state ← [main_shard_state, sub_shard_state]
else
  node_state ← [sub_shard_state]
end if

```

b) *Event handler*: For three messages: propose, preendorse, and endorse, we check if nodes send the message from the same shard as the received nodes. If it is, nodes handle messages as described in II-B. For event wake up, nodes handle as in Alg. 4.

C. Heterogeneous Sharding with the main shard

While Tenderbake implements two successive voting phases, resulting in many sent messages, Emmy plus requires fewer messages. So Emmy plus can better suit shard with few nodes like the main shard, which leaders of each shard form. So in this subsection, we present the **Heterogeneous Sharding**

Algorithm 4 Wake up event algorithm of Classical Sharding with the main shard

```

if is_proposer_main_shard and is_proposer_sub_shard then
  broadcast main_shard_proposal;
  broadcast sub_shard_proposal;
else if is_proposer_sub_shard then
  broadcast sub_shard_proposal;
else if is_leader then
  decide head;
  increment round;
else
  increment round;
end if

```

with the main shard running Emmy plus on the main shard and Tenderbake on sub shard.

a) *Node state*: The leaders in this sharding approach have to maintain two states: the state of the main shard, which runs Emmy plus, and the state of sub shard where Tenderbake is used. The node state is defined as in Alg. 5:

Algorithm 5 Node state of Heterogeneous Sharding with the main shard

```

main_shard_state:
id = node_id,
shard_id = -1,
chain = [],
priority = node_id,
endorsement = [],
sub_shard_state:
id = node_id,
shard_id = node_id / (total_nodes / num_shards),
chain = [],
proposal_state = No_proposal
node_state ← []
if is_leader then
  node_state ← [main_shard_state, sub_shard_state]
else
  node_state ← [sub_shard_state]
end if

```

b) *Event handler*: For this consensus algorithm, we have totally six types of message events: three of Tenderbake, which are proposed, preendorse, endorse, which runs on sub shard, and two for Emmy plus: new_endorsement, new_chain running on the main shard, and a wake-up call event. The way nodes handle the Tenderbake message is the same as II-B. For two message types of the main shard, nodes handle as described in II-A. Finally, for wake-up event, nodes handle as in Alg. 6.

Algorithm 6 Wake up event algorithm

```
if is_leader and not is_proposer_sub_shard then
  if valid_bake_time then
    make a new_block;
    broadcast main_shard_chain with new_block;
  end if
else if is_leader and is_proposer_sub_shard then
  if valid_bake_time then
    make a new_block;
    broadcast chain with new_block;
    broadcast sub_shard_proposal;
  else
    broadcast sub_shard_proposal;
  end if
else if is_proposer_sub_shard then
  broadcast sub_shard_proposal;
else
  increment round;
end if
```

V. PERFORMANCE EVALUATION

A. Experimental Setup

In this section, we evaluate the performance of our proposed algorithms in many aspects. We compare five algorithms by considering three features:

- Throughput measures the number of transactions/records submitted and stored every second.
- Overhead shows the number of messages sent.
- Blocks per hour measures the time taken by the transaction to record, submit and store in the blockchain.

We run the algorithms to measure throughput, overhead, and blocks per hour, and we vary in the number of nodes and shards. We calculate these statistics by fixing the number of iterations to 5000, which shows the total times nodes handle messages. Moreover, we use a Tezos' Blockchain simulator [9] developed and supported by the Tezos community to simulate the Tezos blockchain and test consensus algorithms.

B. Performance Analysis

1) *Throughput*: Fig.5 shows the performance of five approaches when the number of nodes changes. We can see that the throughput of proposed sharding algorithms slightly increases while Emmy plus perform worse. It illustrates that proposed sharding algorithms are more efficient than non-sharding ones when we have a larger quantity of nodes.

In Fig.6, we discuss the transaction per second(TPS) or throughput of three sharding algorithms when the number of shards changes. It reveals that **Heterogeneous Sharding with the main shard** and **Classical Sharding with the main shard** performs better than **Classical Sharding without the main shard** method. This is because the **Classical Sharding without the main shard** requires direct message exchange resulting in a vast number of messages sent between shards. Therefore, the efficiency of **Classical Sharding without the**

main shard is significantly reduced. While in sharding approaches with main shard, nodes can communicate with nodes from other shards via the leaders of each shard.

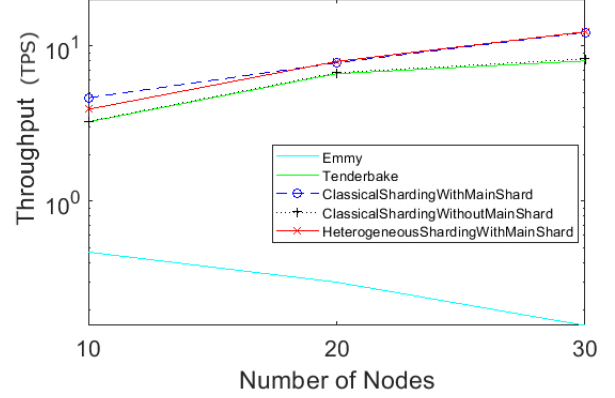


Fig. 5: Throughput with the number of nodes.

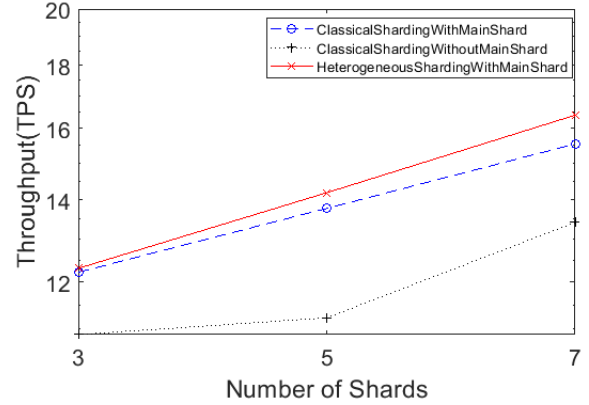


Fig. 6: Throughput with the number of shards.

2) *Overhead*: From Fig.7, we can see that the number of messages sent by all five algorithms decreases rapidly when the number of nodes increases. We can also observe that Emmy plus require the least messages while nodes with Tenderbake need more messages to reach a consensus state. The reason is that Tenderbake implements a three-phase PBFT protocol: propose, preendorse and endorse while Emmy plus requires only one type of message: endorse message.

3) *Blocks per hour*: Fig. 8 shows the blocks per hour of five algorithms with the number of nodes. It is obvious that the blocks per hour of Emmy plus decrease significantly while other algorithms running Tenderbake perform better with the increasing node size. Especially **Heterogeneous Sharding with the main shard**, which runs Emmy plus on the main shard, and Tenderbake on sub shard, outperforms other algorithms with around 1800 blocks per hour.

In Fig.9 we discuss the blocks per hour of three sharding algorithms with the number of shards. We can see that the **Heterogeneous Sharding with the main shard** performs

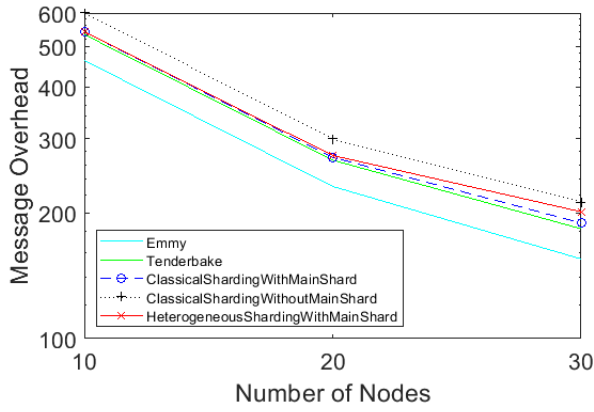


Fig. 7: Number of messages with the number of nodes.

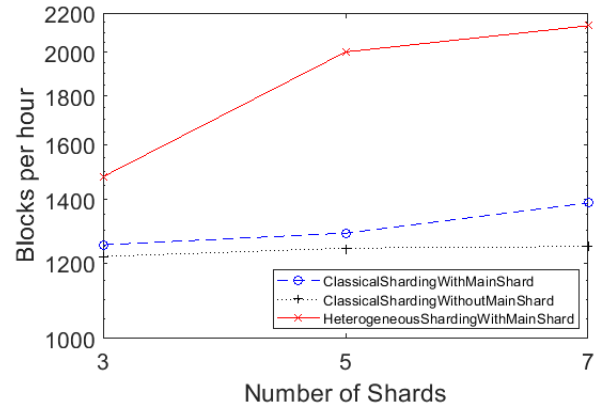


Fig. 9: Blocks per hour with number of shards.

much better than **Classical Sharding without the main shard** and **Classical Sharding with the main shard** with the increasing in shard number. This is because **Heterogeneous Sharding with the main shard** running Emmy plus on main shard has the validation latency which measures the time for validating and adding the blocks to the chain less than Tenderbake (2s to 3s).

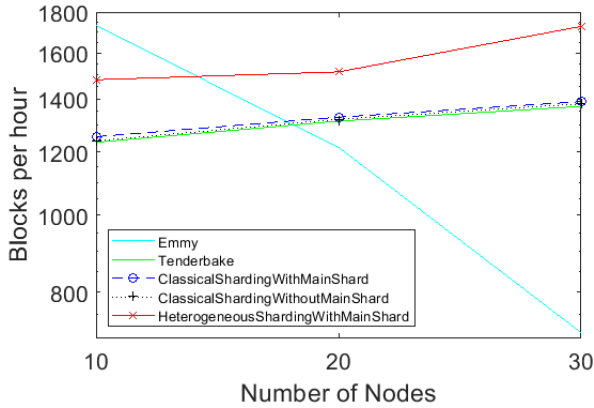


Fig. 8: Blocks per hour with number of nodes.

VI. CONCLUSION

This paper focuses on a sharding-based blockchain system to meet some IoT requirements. Besides, we provide a deep analysis of the performance of non-sharding and sharding-based blockchain networks related to throughput, overhead, and block per hour. We also discussed two sharding possible implementations, and we highlighted the advantage of implementing a heterogeneous sharding-bases consensus algorithm. Experimental results illustrate that the proposed solution increases 54 percent of throughput compared to benchmarks.

In the proposed consensus algorithms, we take into account Emmy plus, Tenderbake algorithms and apply sharding based on these two algorithms. In the future, on the premise of **Heterogeneous Sharding with the main shard** we plan to

implement an adaptive consensus algorithm blockchain system in order to allow greater flexibility and a better adaptation to the different contexts.

ACKNOWLEDGMENT

The authors would like to thank Sara Tucci from CEA LIST, University of Paris-Saclay, France, who participated in the preliminary stages of this project.

REFERENCES

- [1] I. Yaqoob, K. Salah, R. Jayaraman, and Y. Al-Hammadi, "Blockchain for healthcare data management: opportunities, challenges, and future recommendations," *Neural Computing and Applications*, pp. 1–16, 2021.
- [2] V. B. Mišić, J. Mišić, and X. Chang, "Coping with smartly malicious leaders: Pbft with arbitration for blockchain-based iot applications," in *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2021, pp. 1–6.
- [3] A. Hafid, A. S. Hafid, and M. Samih, "Scaling blockchains: A comprehensive survey," *IEEE Access*, vol. 8, pp. 125 244–125 262, 2020.
- [4] H. Nam Nguyen, H. Anh Tran, S. Fowler, and S. Souihi, "A survey of blockchain technologies applied to software-defined networking: Research challenges and solutions," *IET Wireless Sensor Systems*, vol. 11, no. 6, pp. 233–247, 2021.
- [5] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 17–30.
- [6] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 931–948.
- [7] J. T. Victor Allombert, Mathias Bourgoïn, "Introduction to the tezos blockchain," 2019.
- [8] Nomadic Labs. Emmy+: an improved consensus algorithm. Accessed May, 20, 2022. [Online]. Available: <https://research-development.nomadic-labs.com/emmy-an-improved-consensus-algorithm.html>
- [9] L. Aştefanoaei, P. Chambart, A. Del Pozzo, T. Rieutord, S. Tucci, and E. Zălinescu, "Tenderbake—a solution to dynamic repeated consensus for blockchains," *arXiv preprint arXiv:2001.11965*, 2020.
- [10] M. N. S. H. Gang Wang, Zhijie Jerry Shi, "Sok: Sharding on blockchain," *AFT '19, Zurich, Switzerland*, p. 4, 2019.
- [11] M. A. V. S. Z. Z. T. L. B. H. F. H. R. L. Yizhong Liua, Jianwei Liua, "Building blocks of sharding blockchain systems: Concepts, approaches, and open problems."